

SPRINT 4

TAREA: CONFIGURAR EL BACKEND

Configuración del Backend con Spring Boot, PostgreSQL y Conexión Frontend (Reto adicional)

Duración estimada: 2 horas

OBJETIVO DE LA TAREA

Implementarás la persistencia de datos y la integración completa de tu aplicación. Como reto adicional conectaras el frontend previamente desarrollado, con un servidor backend en Spring Boot y una base de datos PostgreSQL.

Deberás crear una base de datos en PostgreSQL, construir una API REST completa en Spring Boot para gestionar tareas y habilitar la comunicación CORS para conectar la interfaz de usuario con tus endpoints.

Resultado esperado

Al finalizar esta actividad tu proyecto deberá:

- Contar con una base de datos PostgreSQL en funcionamiento local para almacenar las tareas.
- Exponer endpoints REST funcionales (GET, POST, PUT, DELETE) desde Spring Boot.
- Reflejar las operaciones del usuario (crear, listar, editar y eliminar) de forma persistente en la base de datos.
- Responder a las peticiones del frontend sin errores de CORS. **RETO ADICIONAL**

Tecnologías y herramientas

- Java (JDK)
- Spring Boot (Spring Web, Spring Data JPA, Validation)
- PostgreSQL & pgAdmin
- JavaScript / Fetch API
- Postman

Desarrollo de la actividad

Paso 1. Configurar la Base de Datos en PostgreSQL

1.1 Crear la base de datos

Abre pgAdmin o la terminal interactiva psql y crea una nueva base de datos para el proyecto:

```
CREATE DATABASE tasks_db;
```

1.2 Registrar parámetros de conexión

Asegúrate de anotar las credenciales de tu servidor local (host, puerto, usuario y contraseña), las cuales usarás en la configuración de Spring Boot.

Paso 2. Inicializar y Configurar el Proyecto en Spring Boot

2.1 Crear el proyecto

Genera un proyecto Spring Boot utilizando Spring Initializr con las siguientes dependencias:

- **Spring Web:** Para la creación de la API REST.
- **Spring Data JPA:** Para el mapeo objeto-relacional (ORM).
- **PostgreSQL Driver:** Conector JDBC para PostgreSQL.
- **Validation:** Para restringir campos vacíos o nulos en las entidades.

2.2 Configurar el archivo application.properties

Edita el archivo src/main/resources/application.properties para definir la conexión con la base de datos recién creada

Paso 3. Crear la Entidad Task y el Repositorio

3.1 Crear la clase Entidad

Define la clase Task en el paquete correspondiente y aplica las anotaciones JPA y Validation necesarias.

Tabla de restricciones del modelo:

Campo	Tipo de Dato	Anotación / Validación
id	Long	@Id, @GeneratedValue(strategy = GenerationType.IDENTITY)
name	String	@NotBlank(message = "El nombre no puede estar vacío")

Campo	Tipo de Dato	Anotación / Validación
description	String	Opcional / Texto descriptivo
dueDate	LocalDate / String	@NotNull(message = "La fecha de entrega es requerida")
status	String	@NotBlank (Ej: PENDING, IN_PROGRESS, DONE)

3.2 Crear la interfaz de Repositorio

Crea la interfaz TaskRepository extendiendo de JpaRepository<Task, Long> para obtener las operaciones CRUD básicas sin implementar SQL manualmente.

Paso 4. Construir los Endpoints REST

Crea un controlador REST (TaskController) expuesto en la ruta base /api/tasks con las siguientes rutas:

Método HTTP	Endpoint	Descripción
GET	/api/tasks	Recupera todas las tareas registradas en la base de datos.
POST	/api/tasks	Crea una nueva tarea enviada en el cuerpo del request.
PUT	/api/tasks/{id}	Actualiza los datos o estado de una tarea existente por su ID.
DELETE	/api/tasks/{id}	Elimina una tarea específica por su ID.

Paso 5. Habilitar CORS y Conectar el Frontend (RETO ADICIONAL)

5.1 Habilitar peticiones Cross-Origin

Permite que el navegador acepte peticiones de tu interfaz web hacia la API. Agrega la anotación `@CrossOrigin` a nivel de clase en el controlador o define un bean de configuración global:

```
@CrossOrigin(origins = "**") // O especifica la URL de tu frontend, ej: http://127.0.0.1:5500
```

5.2 Conectar las peticiones JavaScript con el Backend

Modifica el archivo JavaScript de tu proyecto frontend para realizar solicitudes a la API backend en lugar de manipular únicamente el estado local:

- **Cargar tareas:** Ejecuta una petición GET `http://localhost:8080/api/tasks` al iniciar la página para renderizar la lista desde la base de datos.
- **Guardar tarea:** Tras pasar las validaciones del formulario, envía los datos mediante un POST `http://localhost:8080/api/tasks` con headers: `{ 'Content-Type': 'application.json' }`.
- **Actualizar tarea:** Realiza una petición PUT `http://localhost:8080/api/tasks/{id}` al cambiar el estado de la tarea.
- **Eliminar tarea:** Realiza una petición DELETE `http://localhost:8080/api/tasks/{id}` al presionar el botón de eliminar.



Prueba tu código

Prueba diferentes escenarios tanto desde un cliente de API como desde la interfaz integrada:

Caso 1 - Pruebas directas de API (Postman / cURL)

Envía peticiones GET, POST, PUT y DELETE directamente a `http://localhost:8080/api/tasks`. Confirma el código de respuesta 200/201 HTTP y verifica en pgAdmin que los registros cambien en la base de datos PostgreSQL.

Caso 2 - Creación e Integración desde el Frontend – (si se completa el reto adicional)

Completa el formulario en el navegador y haz clic en enviar. Resultado esperado: La tarea debe enviarse al backend, persistirse en PostgreSQL y mostrarse inmediatamente en la lista.

Caso 3 - Persistencia de Datos (si se completa el reto adicional)

Reinicia la aplicación backend de Spring Boot y recarga la página frontend. Resultado esperado: Las tareas previamente creadas no deben desaparecer, confirmando la persistencia correcta en PostgreSQL.

Checklist de verificación

Verifica cada punto antes de considerar finalizada la tarea.

PostgreSQL & Base de Datos

- ☐ Base de datos tasks_db creada localmente.
- ☐ Archivo application.properties configurado correctamente con credenciales.
- ☐ Tabla generada/mapeada de forma automática sin errores de conexión.

Spring Boot & REST API

- ☐ Entidad Task y TaskRepository contruidos adecuadamente.
- ☐ Endpoints GET, POST, PUT y DELETE probados y funcionales.
- ☐ Validaciones con anotaciones JPA/Validation operativas.

Integración Frontend – RETO ADICIONAL

- ☐ CORS configurado correctamente en Spring Boot.
- ☐ Peticiones Fetch integradas en el JavaScript del frontend.
- ☐ Operaciones del usuario reflejadas correctamente en la base de datos y la interfaz.

RESULTADO ESPERADO

Habrás conectado la interfaz web del Planificador de Tareas con una arquitectura profesional cliente-servidor, logrando:

- ✓ Un servidor Spring Boot estructurado bajo arquitectura REST.
- ✓ Persistencia de datos real utilizando una base de datos PostgreSQL.
- ✓ Integración y sincronización en tiempo real entre el Frontend y el Backend (resultado con reto adicional).